

Discrete Neural algorithmic reasoning

Архитектура воспроизводит пошаговое выполнение алгоритмов на графах помощью механизмов hard graph attention [4]. С учётом того, что любое представление входных данных может быть сведено к графовому виду, модель легко применяется к любой алгоритмической задаче.

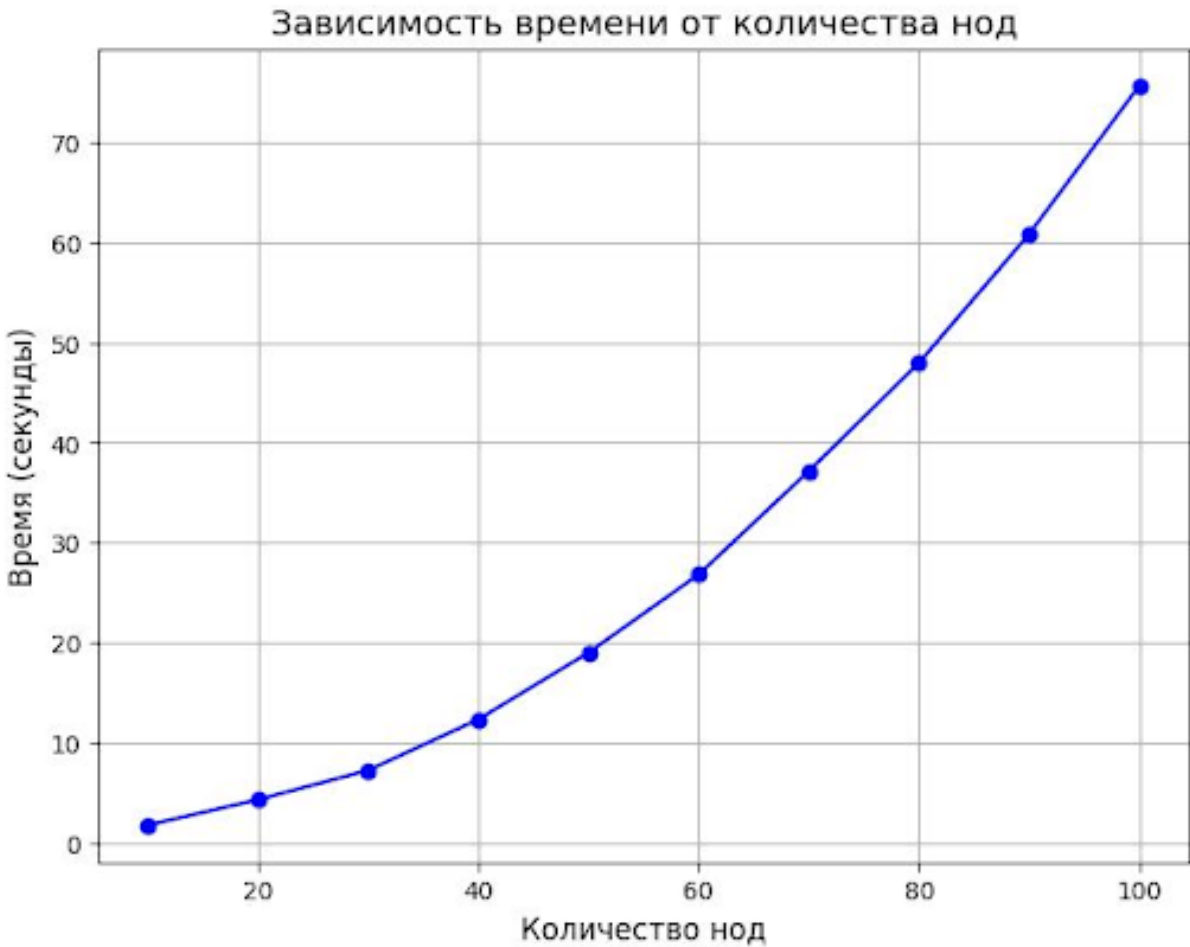
Преимущество модели по отношению к обычным методам состоит в том, что архитектура не чувствительна к размерности задачи, что даёт возможность проводить обучение на задачах меньшей размерности и проводить inference за полиномиальное время в силу устройства архитектуры.

Эксперименты и задачи

Графовые задачи

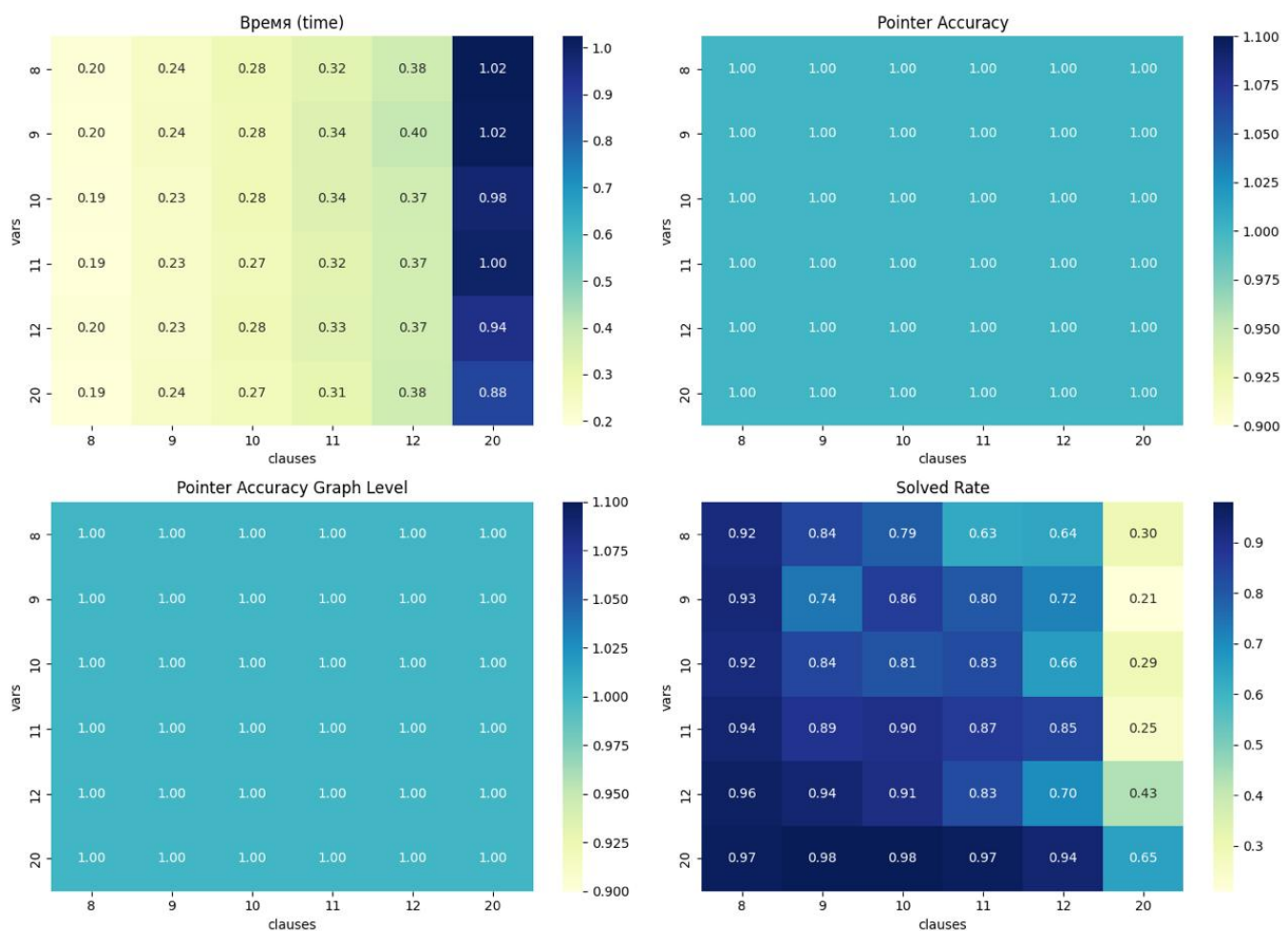
В процессе исследования рассмотрено применение архитектуры к задаче поиска кратчайшего пути, а именно посредством воспроизведения алгоритма Дijkstra.

|                              |     |
|------------------------------|-----|
| pointer_accuracy             | 1.0 |
| pointer_accuracy_graph_level | 1.0 |



Неграфовые задачи

Также рассмотрено применение к неграфовым задачам на примере задачи SAT. Мы использовали сведение этой задачи к задаче поиска максимального независимого множества. Эту задачу мы решали рандомизированным алгоритмом Луби.



Репозитории и исходный код

DNAR + MILP

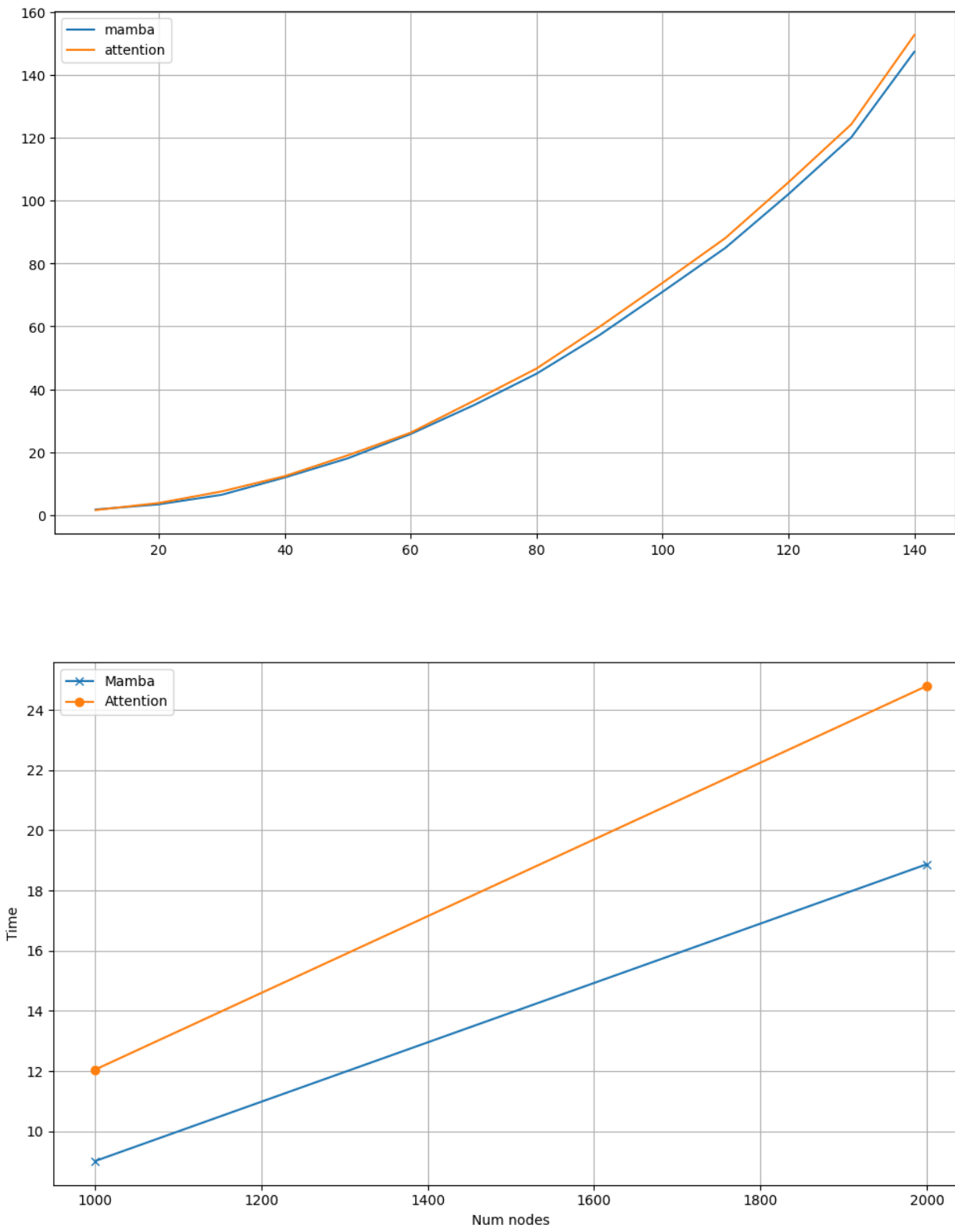


AutoSAT



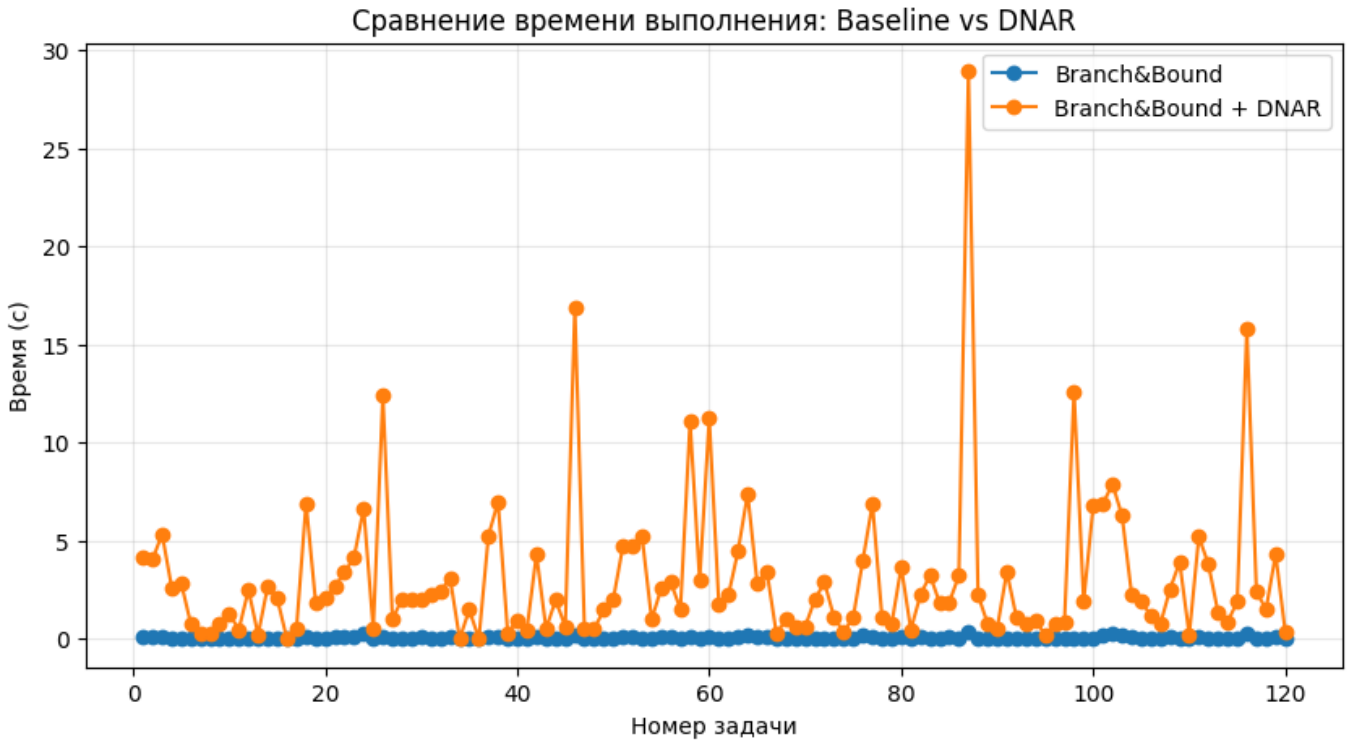
Оптимизация по времени

В контексте задач дискретной оптимизации важно оптимизировать время работы. Поэтому мы реализовали версию архитектуры, в котором квадратичный Attention был заменен линейной Mamba [2]. Добились улучшения по времени, которое особенно заметно на задачах больших размерностях.



Mixed integer linear programming problem

Рассмотрено применение к задаче MILP. Была рассмотрена версия алгоритма Branch&Bound [3] с одновременным использованием стратегий Depth-first search и ветвления по 'наиболее дробной' переменной. Без добавления шагов LP-солвера (релаксация к которому является частью этого алгоритма) в обучающий датасет, архитектура не продемонстрировала способность к обучению. Удалось применить архитектуру для выбора переменной ветвления в комбинации с внешним lp-солвером в роли оракула. Преимуществ по сравнению с обычным подходом такая комбинация не представляет (сравнение по времени на случайных задачах показано на графике). Однако остаётся возможность для обобщения архитектурой большого множества известных эвристик для Branch&Bound (например применение наиболее оптимальной эвристики в зависимости от задачи) – данная часть в процессе исследования. Также есть возможность использования архитектуры как первого приближения решения задачи.



AutoSAT

Алгоритм итеративно оптимизирует эвристики солвера. После каждой итерации LLM получает фидбэк о предложенной ею эвристике: как изменилось время работы солвера после смены эвристики на новую. Показано, что несмотря на значимые улучшения на тренировочной выборке, предложенные моделями эвристики нерелевантны вне тренировочной выборки.

Таблица результатов

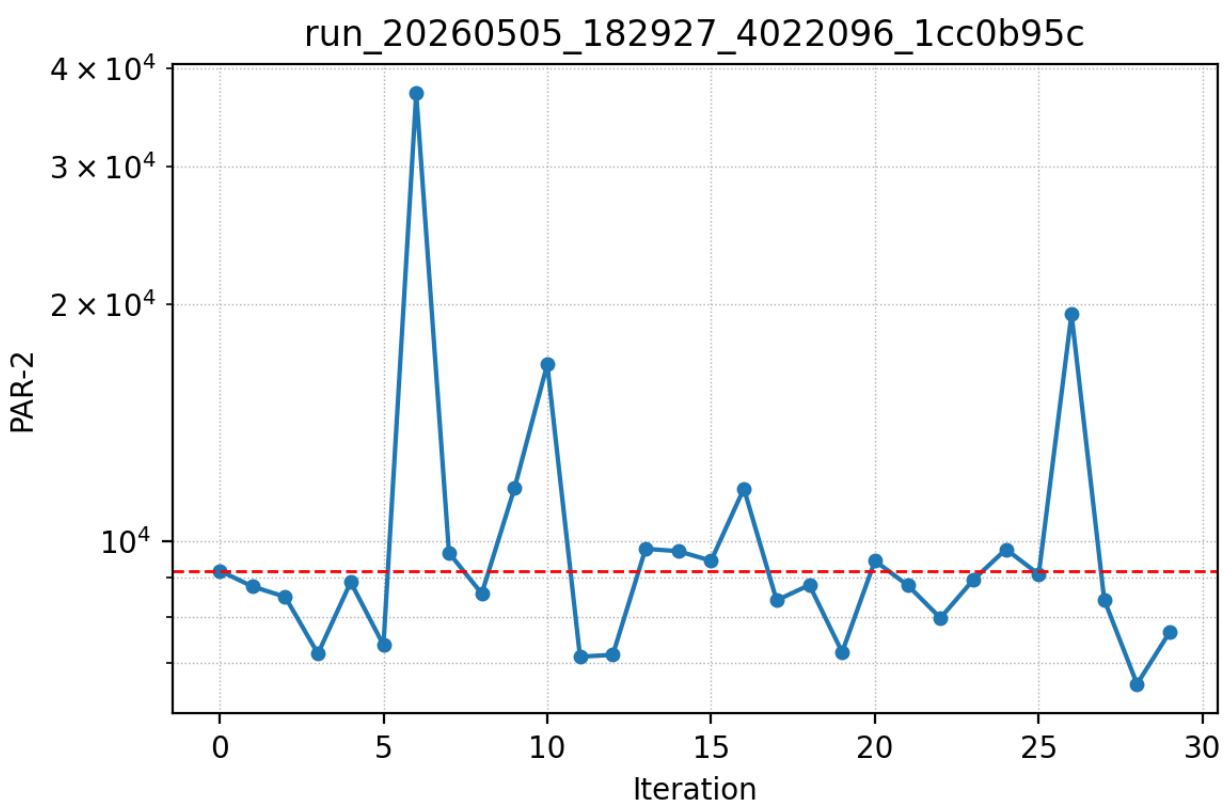
| LLM           | Temperature | Train diff ↓ | Eval diff ↓ |
|---------------|-------------|--------------|-------------|
| DeepSeek v3.1 | 0.9         | -27%         | +4%         |
| DeepSeek v3.1 | 0.8         | -8%          | +3%         |
| DeepSeek v3.1 | 0.6         | -20%         | +3%         |
| gpt-oss-120b  | 0.9         | -15%         | +4%         |
| gpt-oss-120b  | 0.6         | -17%         | +7%         |

Таблица 1. Сравнение лучших результатов, показанных различными моделями

Изменение эвристики с помощью LLM

| Старый код                                                                                                                                                                                                                               | Новый код                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>void Solver::bump_var(int var, double coeff) { if ((activity[var] += var_inc * coeff) &gt; 1e100) { for (int i = 1; i &lt;= vars; i++) activity[i] *= 1e-100; var_inc *= 1e-100;} if (vsids.inHeap(var)) vsids.update(var); }</pre> | <pre>void Solver::bump_var(int var, double coeff) { double inc = var_inc * coeff; activity[var] += inc; if (conflicts &gt; 0) { double conflict_rate = (double)conflicts / (trail.size() + 1); if (conflict_rate &gt; 0.05) { inc *= 1.2; } } if (level[var] &gt; 0) { double age_factor = 1.0 / (1.0 + 0.01 * conflicts); activity[var] *= age_factor; } if (activity[var] &gt; 1e100) { double scale = 1e-100; for (int i = 1; i &lt;= vars; i++) activity[i] *= scale; var_inc *= scale; } if (vsids.inHeap(var)) vsids.update(var); }</pre> |

Визуализация процесса обучения



Литература

[1] M. Ghaffari. An improved distributed algorithm for maximal independent set, 2015.

[2] A. Gu and T. Dao. Mamba: Linear-time sequence modeling with selective state spaces, 2024.

[3] L. Huang, X. Chen, W. Huo, J. Wang, F. Zhang, B. Bai, and L. Shi. Branch and bound in mixed integer linear programming problems: A survey of techniques and trends, 2021.

[4] G. Rodionov and L. Prokhorenkova. Discrete neural algorithmic reasoning, 2025.

[5] Y. Sun, F. Ye, Z. Chen, K. Wei, and S. Cai. Automatically discovering heuristics in a complex sat solver with large language models. 2025.

[6] Y. Sun, F. Ye, X. Zhang, S. Huang, B. Zhang, K. Wei, and S. Cai. Autosat: Automatically optimize sat solvers via large language models. *Journal of Artificial Intelligence Research*, 1(11):1–37, 2024.

[7] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need, 2023.

